ZPDPatch: Execution-Guided Program Repair from Student Submission Trajectories

ANONYMOUS AUTHOR(S)

Automated program repair for programming assignments typically treats a student's latest submission as an isolated buggy program. This snapshot omits the revisions, repeated failures, and partial improvements that led to the current code. We present ZPDPatch, a repair framework that converts authentic submission trajectories into an execution-ordered portfolio of three repair policies. We formalize verdict and testcase improvements as nested execution-evidence orders and prove finite monotone supervision, policy-event inclusion, observed-test non-regression, and pointwise minimal escalation. The Progress adapter learns testcase-level Pareto improvements, Strict learns coarse-verdict improvements, and Answer maps failed submissions to the terminal accepted program. At inference, all policies condition independently on the authentic trajectory; execution stops at the first accepted candidate and otherwise selects the highest pass rate with minimum AST edit distance. We evaluate Qwen2.5-Coder-7B-based adapters on 997 held-out trajectories from 328 test problems drawn from a 492-problem Seen pool, plus 250 trajectories from 54 problem-held-out problems. On seen problems, ZPDPatch improves repair rate from 30.7% for a trajectory-prompted zero-shot baseline to 59.5%, while producing smaller successful patches; a same-problem retrieval baseline reaches 80.2% but makes substantially larger changes. On unseen problems, the corresponding execution-guided system substantially outperforms zero-shot. Across three independent training seeds, Seen and Unseen repair rates average 58.7% and 72.5%, with standard deviations of 1.0 and 1.9 points. Problem-clustered inference preserves both gains. Controlled target-matched and portfolio ablations localize the gain to trajectory-derived relational supervision and heterogeneous policy composition rather than prompt length alone.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Applied computing** → *Education*; • **Computing methodologies** → *Natural language generation.*

Additional Key Words and Phrases: automated program repair, programming education, large language models, submission trajectories, execution-guided repair

ACM Reference Format:
Anonymous Author(s). 2027. ZPDPatch: Execution-Guided Program Repair from Student Submission Trajectories. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (FSE '27).* ACM, New York, NY, USA, 20 pages.

1 Introduction

Programming students iteratively write, submit, execute, and revise code. Online judges make this loop scalable, but their feedback is usually limited to coarse verdicts such as `Wrong Answer`, `Runtime Error`, or `Time Limit Exceeded`. A verdict reports failure without explaining whether the latest revision made progress, repeated an earlier defect, or moved away from a viable solution. Instructors cannot inspect every trajectory and author an individualized repair at the scale of modern programming courses.

Automated program repair (APR) offers executable, code-specific feedback. Systems for programming assignments cluster peer solutions, align a student's code to a reference, or synthesize a fixed program [Gulwani et al. 2018; Hu et al. 2020; Wang et al. 2018]. More recent systems use language models, execution diagnostics, retrieved examples, and iterative conversations [Joshi et al. 2023; Liu et al. 2024; Orvalho et al. 2025; Yang et al. 2024; Zhang et al. 2024]. Most of these approaches condition on the latest buggy program, possibly augmented with tests or peer programs. The earlier submissions produced by the same student are rarely treated as first-class repair context.

The motivation for using history is straightforward but unverified. Two students may arrive at similar current programs through different revisions. Their histories expose repeated errors, attempted edits, and states that they demonstrably reached. In educational terms, such reachable improvements resemble the Zone of Proximal Development (ZPD): states attainable with appropriate support but not yet reached independently [Chounta et al. 2017; Cole et al. 1978; Murray and Arroyo 2002]. We therefore ask whether repair can exploit a student's observed trajectory to produce a useful next program rather than immediately replacing it with an arbitrary correct solution. We use ZPD as a design motivation, not as a measured psychological construct or a claim about learning outcomes.

We introduce ZPDPatch, illustrated in Figure 1. It automatically converts authentic submission trajectories into three supervised repair datasets. Progress retains verdict improvements and same-verdict transitions whose per-test outcomes improve monotonically. Strict retains only transitions that improve the online-judge verdict. Answer pairs every failed submission with the trajectory's final accepted program. Three QLoRA adapters are trained independently over the same code language model and prompt. At inference time, ZPDPatch calls the adapters from narrower to broader supervision, executes each candidate, stops on acceptance, and otherwise selects a non-regressive candidate using test pass rate and AST tree edit distance (TED).

Our evaluation separates three mechanisms that are easy to conflate: learned relational targets, heterogeneous policy composition, and the serialization of prior submissions. The complete system substantially improves over zero-shot on both seen and problem-held-out data, and sequential escalation outperforms each adapter alone. Target-matched retraining further shows that the learned benefit survives removing earlier submissions from the inference prompt. Thus, trajectories contribute not merely extra prompt tokens but automatically mined reachability relations that are distilled into policy parameters; independent-policy composition then turns those relations into complementary executable candidates.

This paper makes the following contributions:

- We formulate trajectory-mined repair as nested execution-evidence orders plus terminal closure, prove an end-to-end safety theorem from finite monotone supervision, policy inclusion, non-regression, and minimum-breadth escalation, and provide executable audits of these properties.
- We design an execution-guided repair procedure that composes independently conditioned policies, supports early stopping, and abstains when no candidate improves over the current program.
- We conduct a problem-familiarity-aware evaluation against zero-shot and retrieval-based repair, including target-matched, repeated-policy, generated-feedback, ordering, multi-seed, and problem-clustered analyses.

2 Background and Related Work

2.1 Structure- and Reference-Based Educational Repair

Educational APR can exploit many programs written for one specification. CLARA clusters and aligns programs before extracting a repair; SARFGEN searches and aligns reference programs; Refactory normalizes refactorings before comparison [Gulwani et al. 2018; Hu et al. 2020; Wang et al. 2018]. These systems provide strong assignment-specific evidence and can preserve a recognizable strategy when an appropriate peer cluster exists. Work on multi-function, neural, and diversified feedback broadens the target beyond one aligned correction [Choi et al. 2021; Choi and Lee 2025; Heo et al. 2023; Yi et al. 2017]. Together, they establish that execution correctness and relationship to the student's program are separate repair properties.

LSGen is our reference-rich baseline. It filters historical repair pairs, retrieves by edit information, generates, and may retrieve again from the new state [Dai et al. 2026]. We retain its access to other users' Seen-Train solutions for the target problem. That information is unavailable on genuinely new assignments, so LSGen is not evaluated on problem-held-out Unseen tasks. This separates repository-mature repair from repair using only a learned model, statement, and tests.

2.2 Neural and Large-Language-Model Repair

RING, PyDex, and FastFixer cast repair as conditional code generation, reducing dependence on fixed transformation libraries [Joshi et al. 2023; Liu et al. 2024; Zhang et al. 2022, 2024]. Retrieval and localization can constrain generation: peer-aided repair supplies other students' programs, and high-precision syntax feedback narrows the requested change [Phung et al. 2023; Zhao et al. 2024]. The resulting systems cover programs that do not align cleanly with a reference, but make correctness and patch scope external validation concerns.

Execution-aware methods address those concerns with signals independent of model confidence. SelfAPR learns from test diagnostics; CREF carries diagnostics through a repair conversation; counterexample-guided repair combines generation, fault localization, and MaxSAT [Orvalho et al. 2025; Yang et al. 2024; Ye et al. 2022]. ZPDPatch likewise executes candidates, but introduces a different unit of supervision and control. Authentic trajectories define nested improvement relations rather than synthetic corruptions or manually assigned repair roles. Independently trained members share a base model, generated candidates remain conditionally independent given authentic observations, and an explicit selector includes the unchanged program. Thus a failed generation cannot force a regression or contaminate another policy's input distribution.

2.3 Adaptive Feedback, ZPD, and Submission Histories

Programming feedback ranges from correctness reports to next-step hints and complete repairs [Keuning et al. 2018; Narciss 2008]. Tutoring research studies help seeking and assistance at a productive granularity; iSnap and LLM hint systems similarly expose next-step or multi-level support [Aleven and Koedinger 2000; Heickal and Lan 2024; Price et al. 2017; Roest et al. 2024; Xiao et al. 2024]. The ZPD motivates support between independent and assisted performance [Chounta et al. 2017; Cole et al. 1978; Murray and Arroyo 2002]. We operationalize only observed reachability: a later improving submission proves that this user reached that program state, not that showing the edit causes learning or is optimal scaffolding.

Submission sequences also support knowledge tracing, which estimates latent mastery from interaction histories [Corbett and Anderson 1994; Ghosh et al. 2020; Piech et al. 2015]. ZPDPatch instead orders source-code states using execution evidence; it neither infers mastery nor predicts a learner response. This distinction permits fully automatic repair targets while delimiting the contribution to program behavior rather than educational outcomes.

2.4 Positioning of This Work

Table 1 locates the contribution along evidence and composition axes. Unlike reference alignment and LSGen, ZPDPatch does not require a same-problem solution repository; unlike prior LLM and iterative repair, its targets are mined as ordered transitions from the same user's authentic submissions. Its technical contribution is the combination of this execution-evidence order, policy-specific parameterization, and certified sequential selection.

3 Problem Formulation

For problem p, let a student's i-th submission be $s_i = (c_i, v_i)$, where c_i is the complete source code and v_i is the online-judge verdict. A submission trajectory $\tau = [s_1, s_2, \dots, s_n]$ is chronologically ordered and terminates at the first accepted submission. Let d_p contain the problem statement and resource constraints, and let $\mathcal{E}_p$ be the available test suite. Re-executing c_i gives a per-test outcome vector $o_i = (o_i(e))_{e \in \mathcal{E}_p}$.

Table 1. High-level positioning of educational repair approaches. "Same-problem refs." denotes a database of solutions or repair pairs for the target assignment; "trajectory supervision" denotes training targets mined from ordered submissions by the same user.

Approach family	Refs.	Exec.	Traj.	Stages
Structural alignment [Gulwani et al. 2018; Hu et al. 2020; Wang et al. 2018]	Yes	Opt.	No	No
LLM generation [Joshi et al. 2023; Liu et al. 2024; Zhang et al. 2024]	Opt.	Opt.	No	No
Iterative repair [Orvalho et al. 2025; Yang et al. 2024]	Opt.	Yes	No	Yes
LSGen [Dai et al. 2026]	Yes	Yes	No	Yes
ZPDPatch	No	Yes	Yes	Yes

A repair example consists of problem context d_p, an observed sequence h, and a complete target program y. Adapter-specific rules decide which submissions remain in h and which later submission supplies y; the two need not be adjacent in the original trajectory. Given $x = (d_p, h)$, a model generates one complete program $\hat{c}$.

We call a later state *reachable* when it occurs in the same student's observed trajectory and satisfies an adapter-specific improvement rule. Reachability is an empirical property of the data, not a guarantee that the transition is minimal, comprehensible, or caused by instructional support. Our design hypothesis is that learning from such transitions can produce repairs that preserve more of the current solution than direct answer generation.

4 Approach

4.1 Design Requirements

Four requirements shape the method: *automatic supervision* recoverable from submissions and tests; *target diversity* spanning intermediate improvement and accepted solutions; *executable validation* independent of model confidence; and *non-regression* when no candidate improves observed behavior. We separate these offline and online decisions. Label relations determine what each adapter learns; execution and selection determine what the system may return. A noisy target can influence generation without automatically overriding a better current program.

4.2 Trajectory Construction

We group submissions by problem and student, sort by time, and stop at first acceptance. After indentation/comment/blank-line normalization, we remove only consecutive duplicates; non-consecutive revisits remain evidence of distinct attempts. Complete source, verdict, and order are preserved, with no maximum history or transition count. Every state is re-executed. Cache keys bind problem, normalized code, test-suite identity, timeout, and memory limit, preventing incompatible reuse. Runner failures receive an explicit most-severe outcome, and Progress construction requires a complete cache.

4.3 Adapter-Specific Supervision

We order verdict severity as $\mathrm{sev}(\texttt{AC}) = 0$, $\mathrm{sev}(\texttt{WA}) = 1$, $\mathrm{sev}(\texttt{TLE/MLE}) = 2$, $\mathrm{sev}(\texttt{RE}) = 3$, $\mathrm{sev}(\texttt{CE}) = 4$, and 5 for internal failures. For equal, non-empty test keys, $o_t \preceq_{\mathrm{sev}} o_r$ iff no test outcome in o_t is more severe than its counterpart in o_r. Test-case progress is the Pareto condition

$$\mathrm{TCProg}(r, t) = \mathbf{1}[o_t \preceq_{\mathrm{sev}} o_r \wedge o_t \neq o_r]\,. \tag{1}$$

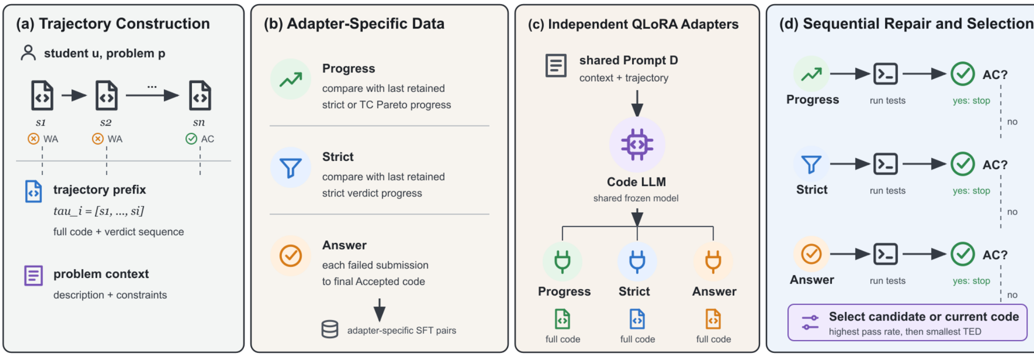

Fig. 1. Overview of ZPDPatch. Authentic submission trajectories are normalized and re-executed. Adapter-specific rules construct supervision for Progress, Strict, and Answer. Independently trained adapters generate candidates sequentially; execution outcomes determine early stopping and final selection.

Answer. If s_n is accepted, every preceding non-accepted submission is independently paired with c_n. For $\tau = [S_1, \ldots, S_9]$, the rule produces $([S_1], c_9), \ldots, ([S_8], c_9)$. Thus, Answer learns a direct failed-program-to-answer mapping and receives no multi-submission history.

Strict. Starting with $R = [s_1]$, we scan the trajectory and append s_t only when its verdict is strictly better than the last retained submission $s_r = R[-1]$: $\mathrm{sev}(v_t) < \mathrm{sev}(v_r)$. If $R = [r_1, \ldots, r_m]$, we create $([r_1, \ldots, r_{k-1}], c_{r_k})$ for every $k = 2, \ldots, m$. The comparison is against the last retained state, not necessarily the immediately preceding original submission.

Progress. Progress uses the same retained scan but appends s_t when either its verdict strictly improves or $v_t = v_r$ and $\mathrm{TCProg}(r, t) = 1$. It therefore captures monotonic per-test progress that is hidden by an unchanged coarse verdict. Prefix-target construction is otherwise identical to Strict.

The datasets are constructed independently and may overlap. A strict transition is also a progress transition, and final accepted transitions can occur in both. The objectives nevertheless differ: Answer learns direct completion from one failed program, whereas Strict and Progress learn transitions from retained prefixes.

Algorithm 1 gives the common retained-prefix construction. The predicate Keep is the only difference between Strict and Progress. Comparing a candidate with the last retained state, rather than the immediately previous raw submission, makes the retained sequence monotonic under the selected rule. Importantly, the algorithm emits every retained prefix. A trajectory with m retained states yields $m - 1$ training examples, so early transitions are not discarded in favor of only the final prefix.

4.4 Execution-Evidence Order

The three rules are not unrelated task labels. They form two nested execution-evidence orders plus a terminal closure. Let the evidence of submission (s) be (e(s)=(v(s),o(s))) over the fixed testcase universe $\mathcal{E}_p$. Define the strict relation

$$s \prec_S t \iff \mathrm{sev}(v(t)) < \mathrm{sev}(v(s)), \tag{2}$$

and the progress relation

$$s \prec_P t \iff s \prec_S t \ \vee \ \left(v(s) = v(t) \wedge o(t) \prec_{\times} o(s)\right), \tag{3}$$

Algorithm 1 Retained-prefix supervision for Strict and Progress

Require: Trajectory $\tau = [s_1, \ldots, s_n]$, policy $a \in \{\text{strict}, \text{prog}\}$
Ensure: Supervised examples D_a

```
1: R ← [s_1]; D_a ← []
2: for t ← 2 to n do
3:     r ← R[−1]
4:     q ← sev(v_t) < sev(v_r)
5:     if a = prog then
6:         q ← q ∨ (v_t = v_r ∧ TCProg(r, t))
7:     end if
8:     if q then
9:         append (R, c_t) to D_a; append s_t to R
10:    end if
11: end for
12: return D_a
```

where $o(t) \prec_\times o(s)$ is the strict product order: every testcase outcome is no worse and at least one is better under severity. Strict and Progress retain a chain under $\prec_S$ and $\prec_P$, respectively. Answer is the terminal closure $A(s) = s_n$ from each failed state to the first accepted state; it is intentionally not an intermediate order.

This view makes the target granularity explicit. Strict observes only a projection of execution evidence onto the coarse verdict. Progress refines that projection with the testcase product order. Answer discards intermediate ordering and connects a failed state to the absorbing accepted state. Adapter specialization therefore corresponds to distinct relations over the same trajectory rather than to role words inserted in prompts.

Proposition 1 (Finite monotone supervision). *For a problem with T tests, every transition retained by Progress strictly decreases the lexicographic rank $\rho(s) = (\text{sev}(v(s)), \sum_{e \in \mathcal{E}_p} \text{sev}(o(s,e)))$. Consequently, retained Progress chains contain no cycles and have length at most $6(5T+1)$. Strict chains satisfy the same result through their first rank component.*

Proof. A Strict transition decreases the first component, independently of the second. A same-verdict Progress transition is Pareto non-worsening and strict on at least one testcase, so it preserves the first component and decreases the sum. Thus every retained transition strictly decreases ρ. There are six verdict-severity values and $5T+1$ possible sums, which gives the finite bound. □

Proposition 2 (Policy-event inclusion). *For the same trajectory and initial state, every target event retained by Strict is retained by Progress.*

Proof. Both scans retain every strict verdict improvement. A transition retained only by Progress has the same verdict as its predecessor and therefore does not change the last-retained verdict severity. By induction over the raw trajectory, both scans have equal last-retained verdict severity whenever the next strict improvement occurs, so they retain that event simultaneously. Progress may additionally retain same-verdict Pareto improvements. □

Proposition 3 (Nested assistance horizons). *For a failed state s_i in a trajectory whose first accepted state is s_n, let j_P and j_S be the first later indices satisfying $s_i \prec_P s_j$ and $s_i \prec_S s_j$, respectively. Then both exist and $j_P \le j_S \le n$; Answer's target is exactly s_n.*

Proof. Acceptance has minimum verdict severity, so $s_i \prec_S s_n$ for every failed s_i, establishing existence and $j_S \leq n$. Because $\prec_S \subseteq \prec_P$, every Strict-improving state is also Progress-improving. The first Progress-improving index therefore cannot occur after the first Strict-improving index. □

The proposition derives the assistance breadth used online from target reachability rather than assigning arbitrary policy names: Progress can stop at an earlier observed improvement, Strict waits for coarse-verdict progress, and Answer closes the remaining distance to acceptance. The inequality can be strict or collapse to equality on a particular trajectory; the adapters retain separate supervision distributions in either case.

4.5 Why an Order Rather Than a Scalar Reward?

A scalar such as number of passed tests can declare one submission better even when it breaks a previously passing testcase. Equation 1 deliberately refuses that compensation: within one coarse verdict, Progress accepts only Pareto improvements. Strict permits a change in testcase outcomes only when the judge's coarse verdict itself improves. This creates a lexicographic evidence geometry: coarse failure class is the first coordinate, and non-compensatory testcase severity is the refinement used inside one class. The rank in Proposition 1 proves termination, while the product order preserves which individual observations justify a transition.

The relations also separate *supervision soundness* from *model soundness*. A retained target is sound with respect to its declared evidence order because it was actually executed and observed later; this says nothing about whether a learned model reproduces that relation on a new input. Algorithm 2 closes that gap at inference by executing every generated program and including the original program in selection. The composition therefore has two proof boundaries: Propositions 1–3 certify what may enter training, and Propositions 4–5 certify what the controller may return and how far it escalates. No likelihood score is used as a correctness surrogate.

These properties depend on a fixed testcase universe and severity map. Cache keys consequently bind the testcase identity and resource limits, and Progress fails closed when a complete outcome vector is unavailable. This engineering constraint follows from the theory: comparing vectors from different universes would make the product order undefined, while silently dropping a testcase could turn a regression into an apparent Pareto improvement.

4.6 Illustrative Trajectory

Consider an illustrative trajectory with verdicts

$$[\mathsf{RE}, \mathsf{WA}, \mathsf{WA}, \mathsf{WA}, \mathsf{AC}]$$

and three-test outcome vectors

$$[(\mathsf{RE}, \mathsf{RE}, \mathsf{RE}), (\mathsf{AC}, \mathsf{WA}, \mathsf{WA}), (\mathsf{AC}, \mathsf{AC}, \mathsf{WA}), (\mathsf{AC}, \mathsf{WA}, \mathsf{WA}), (\mathsf{AC}, \mathsf{AC}, \mathsf{AC})].$$

The example is synthetic. Strict retains S_1, S_2, S_5, emitting $([S_1], c_2)$ and $([S_1, S_2], c_5)$. Progress also retains S_3, where one test improves without regression, but rejects S_4, which regresses relative to the last retained state; it emits three prefixes over S_1, S_2, S_3, S_5. Answer emits four independent pairs $([S_1], c_5), \ldots, ([S_4], c_5)$. Thus Progress can preserve useful same-verdict edits, Strict skips them, and Answer can learn from states excluded by both chains. The construction is filtered transition learning, not next-submission imitation: a non-adjacent state can be targeted when intervening submissions violate the relation.

4.7 Prompt and Adapter Training

All adapters share one role-neutral prompt: statement and limits, chronologically retained complete programs with verdict/outcome/edit summaries, and the current program requesting one complete

Python solution. No instruction names a policy or asks for a particular edit size, so specialization can arise only from supervision. Only assistant target tokens contribute to loss. Whole-program targets avoid patch-application failures at the cost of regenerating unchanged context, motivating greedy decoding and early stopping.

For adapter $a \in \{\text{prog}, \text{strict}, \text{ans}\}$, let $\mathcal{D}_a$ be its dataset, x_i the prompt, and y_i the target tokens. We minimize the target-only supervised fine-tuning objective

$$\mathcal{L}_a = -\frac{1}{|\mathcal{D}_a|} \sum_{(x_i,y_i)\in\mathcal{D}_a} \frac{1}{|y_i|} \sum_{k=1}^{|y_i|} \log P_{\theta,\phi_a}(y_{i,k} \mid y_{i,<k}, x_i), \tag{4}$$

where θ is the frozen base model and ϕ_a is an adapter's QLoRA parameter set. Prompt tokens are masked from the loss. The adapters are trained and stored independently.

4.8 Execution-Guided Sequential Repair

At inference, we invoke Progress, Strict, and Answer from fine-grained improvement to coarse improvement to terminal solution. Each adapter receives the same observed student trajectory and produces one complete program; generated candidates are never inserted into another adapter's prompt. This independence makes the three outputs a policy portfolio rather than a synthetic trajectory whose later states may lie outside the training distribution. Execution has two roles only: an accepted candidate stops the sequence, and failed candidates are compared by the certified selector below. The order is fixed before evaluation and tested independently in RQ3–RQ6. We also evaluate the previously plausible alternative of appending generated execution feedback as a controlled ablation.

If all candidates fail, the selector includes the current program c_i as a fallback:

$$C = \{c_i, c^{\text{prog}}, c^{\text{strict}}, c^{\text{ans}}\}. \tag{5}$$

Let $\text{Pass}(c)$ be the fraction of tests passed and $\text{ted}(c_i, c)$ the Python AST tree edit distance. Selection is lexicographic:

$$c^* = \arg\max_{c\in C} \left(\text{Pass}(c), -\text{ted}(c_i, c)\right). \tag{6}$$

The selector first maximizes executed correctness and then minimizes change. If no candidate improves on c_i, the unchanged fallback has TED zero and the system abstains. Algorithm 2 preserves conditional independence of generated candidates given the authentic observed trajectory; all executed candidates remain available to final selection.

4.9 Certified Selection and Computational Cost

Proposition 4 (Observed-test non-regression). *For every input, Algorithm 2 returns a program c^* such that* $\text{Pass}(c^*) \geq \text{Pass}(c_i)$.

Proof. If the algorithm stops early, the returned candidate has pass rate one. Otherwise, c_i is a member of C, and the final selector maximizes pass rate before edit distance. Its maximum therefore cannot be below $\text{Pass}(c_i)$. □

Proposition 5 (Pointwise minimal escalation). *Assume each policy candidate is conditionally independent of policy order given the observed trajectory, and assign breadth* $b(\text{Progress}) < b(\text{Strict}) < b(\text{Answer})$. *Among all permutations with acceptance-based early stopping, Algorithm 2 returns a successful candidate of minimum breadth whenever at least one policy succeeds.*

Proof. The algorithm visits policies in strictly increasing breadth and stops at the first success. Every skipped policy has lower breadth and failed, while every unvisited policy has greater breadth.

Algorithm 2 Execution-guided sequential repair

Require: Problem p, current code c_i, observed trajectory h

$C \leftarrow [c_i]$
for a in [Progress, Strict, Answer] **do**
  $c^a \leftarrow \text{Generate}(a, p, h)$
  $z^a \leftarrow \text{Execute}(p, c^a)$; append c^a to C
  if $\text{Pass}(c^a) = 1$ **then**
    return c^a
  end if
end for
return $\arg\max_{c \in C}(\text{Pass}(c), -\text{ted}(c_i, c))$

The returned successful policy therefore minimizes breadth over the successful set. Any alternative permutation can only choose the same minimum or a broader successful policy. □

Theorem 1 (End-to-end evidence safety). *Under a fixed testcase universe and severity map, the union of Progress, Strict, and Answer supervision edges for any finite first-acceptance trajectory is a directed acyclic graph. Given order-independent policy candidates, ZPDPatch returns (i) a minimum-breadth accepted candidate when one exists, or (ii) a candidate whose observed pass rate is no lower than the current program otherwise.*

Proof. Proposition 1 makes every Progress and Strict edge strictly rank-decreasing. Every Answer edge starts at a failed state and ends at acceptance, so its first rank component strictly decreases to zero. A directed cycle would require rank to return to a larger value and is therefore impossible. The online alternatives follow from Propositions 5 and 4, respectively. □

The guarantees are relative to the observed test suite and to the derived breadth order; they neither prove semantic correctness on hidden inputs nor forbid a large edit when that edit improves more tests. Candidate independence is essential to Proposition 5: feeding one generated program into the next adapter changes the candidate under a counterfactual order. We validate the supervision, selection, and ordering propositions with executable certificates in Section 6.

For a raw trajectory of length n and T tests, dataset construction performs a single retained scan once cached outcomes exist. Comparing outcome vectors costs $O(T)$, giving $O(nT)$ time and $O(n)$ retained-state storage per trajectory, excluding program text. Answer construction is $O(n)$. At inference, generation dominates cost. The system performs between one and three generations and executions, with exactly three only when neither early stage produces an accepted program. The cache can eliminate repeated execution for identical candidates but does not change the worst-case generation count.

5 Experimental Design

5.1 Research Questions

- **RQ1: Repair Effectiveness.** How effectively does ZPDPatch repair held-out trajectories from problems observed during training, compared with zero-shot and retrieval-based repair?
- **RQ2: Trajectory Conditioning.** Does access to the full submission prefix improve repair over training and inference with only the current code?
- **RQ3: Adapter Specialization.** Do Progress, Strict, and Answer exhibit different repair coverage and patch size?

RQ4: Sequential Escalation. Does execution-guided composition improve effectiveness without always invoking all adapters?

RQ5: Problem Generalization. Does ZPDPatch retain an advantage on problems completely excluded from training?

RQ6: Guarantees and Robustness. Do the supervision and selection invariants hold in the complete artifacts, and are conclusions robust to problem-level dependence and adapter ordering?

5.2 Dataset Construction and Splits

We combine Python submissions, statements, metadata, and test cases for problems in Project CodeNet's Python800 benchmark [Puri et al. 2021]. We retain trajectories with at least three submissions, at least one non-accepted state before the first acceptance, and a final accepted program present in the Python800 reference set. We remove post-acceptance submissions and consecutive normalized duplicates. Users must appear in at least three problems and each retained problem must contain at least two trajectories. The refined corpus contains 546 problems, 21,454 trajectories, 99,432 submissions, and 10,088 tests.

Before splitting, we tokenize every possible Answer, Strict, conservative Progress, and final-evaluation prompt-target configuration. If any configuration exceeds 4,096 tokens, we exclude the entire trajectory rather than truncating code or history. This removes 2,896 trajectories and leaves 18,558. No later `max_history`, transition-count, code, or test truncation is applied.

We sort problems by eligible trajectory count and assign the top 90% (492 problems) to *Seen* and the remaining 54 low-resource problems to *Unseen*. Within every Seen problem, we reserve at least one trajectory for Train, Validation, and Test, then obtain a deterministic 80:10:10 trajectory split. All 260 Unseen trajectories remain problem-held-out test data. Table 2 reports the resulting corpus. Users may occur in more than one split; RQ5 is problem-held-out, not user-held-out.

Adapter construction yields the training and validation sizes in Table 3. Final evaluation uses the prefix ending at the last non-accepted submission and the final accepted program as the oracle. We retain only cases for which re-execution confirms the current program as non-accepted and the oracle as accepted, leaving 997 Seen and 250 Unseen repair instances.

5.3 Baselines

Zero-shot. The zero-shot baseline uses the same Qwen base model and prompt as ZPDPatch without fine-tuning. If a candidate fails, its verdict and number of passed tests are added to the conversation before the next attempt. It receives at most three generations, matching ZPDPatch's maximum budget. Because it receives the full trajectory, comparison with zero-shot measures the complete effect of learned adapters and sequential composition, not history in isolation.

LSGen. LSGen is an iterative retrieval-based solution generator [Dai et al. 2026]. We preserve its repair-pair filtering, edit-vector retrieval, diff-guided generation, and re-retrieval, while replacing hard-coded infrastructure with our common dataset and runner. For a Seen query, the retrieval database contains buggy/correct pairs from other Seen-Train trajectories of the same problem; the same student and example are excluded. LSGen retrieves five pairs and performs at most three iterations. We do not evaluate it on Unseen problems because exposing same-problem correct programs would violate problem holdout.

All approaches use the same base LLM, public tests, 2.5-second per-test timeout, 2GB memory limit, and maximum of three candidate generations.

Table 2. Dataset statistics after whole-trajectory context filtering. "Prefix" counts possible raw prefixes before adapter-specific filtering.

Group	Role	Problems	Tests	Traj.	Sub.	Prefix	Users
Seen	Train	492	9,446	14,638	58,872	44,234	4,072
	Validation			1,830	7,283	5,453	1,283
	Test			1,830	7,068	5,238	1,291
Unseen	Test	54	642	260	965	705	237

Table 3. Adapter supervision datasets. Validation examples are excluded from training.

Adapter	Train examples	Validation examples
PROGRESS	21,416	2,685
STRICT	16,973	2,116
ANSWER	40,454	4,965

5.4 Implementation

We use Qwen2.5-Coder-7B-Instruct [Hui et al. 2024] and train each adapter for one epoch with 4-bit QLoRA [Dettmers et al. 2023]. The base weights use NF4 double quantization and BF16 computation. LoRA targets the q, k, v, o attention projections and gate/up/down MLP projections with rank 16, scaling 32, and dropout 0.05. Optimization uses paged 8-bit AdamW, learning rate 2×10^{-4}, cosine scheduling, 0.03 warmup ratio, gradient checkpointing, micro-batches of 2, and eight accumulation steps. Seed 2027 is primary; seeds 2028 and 2029 repeat all adapter trainings and Seen/Unseen evaluation with identical data, hyperparameters, and greedy decoding. We select the lowest validation-loss checkpoint and run on one NVIDIA RTX 5090. OpenAI Codex assisted experiment-script and analysis-code development; all jobs ran on our infrastructure, artifacts were independently checked, and no generated datum or citation was accepted as evidence.

A single Python runner executes original and generated programs. We cache outcomes by problem, code, test set, timeout, and memory configuration and reuse the cache across supervision construction and evaluation. Candidate generation can proceed in parallel, but candidate execution and recorded outcomes are shared so that an identical program cannot receive inconsistent labels under CPU contention.

5.5 Ablations

For RQ2, we construct adapter-specific STRICT and PROGRESS examples for both Seen and Unseen tests. *Full Trajectory* uses each retained prefix. *Current Code Only* retrains an adapter on identical examples and targets after removing every submission before the current one; its displayed position is reset to S_1 to prevent index leakage. ANSWER is excluded because its inputs already contain one submission. This design isolates the presence of history while holding targets, counts, model, and hyperparameters fixed.

For RQ3, each adapter independently generates one candidate for the same 997 Seen instances. For RQ4, we compose these candidates with PROGRESS–STRICT–ANSWER early stopping and fallback selection. Two controls distinguish policy diversity from repeated use and prompt-state effects: *Answer*×3 invokes the same Answer adapter up to three times, appending generated execution feedback so that deterministic greedy decoding can produce a non-identical later candidate; repeating the identical prompt would collapse exactly to Answer once. *Generated Feedback* applies the same update protocol to the heterogeneous P–S–A policies. The final *Independent Policies* system instead keeps every policy conditioned on the same authentic trajectory. All configurations share the maximum three-generation budget, execution limits, and fallback selector. Answer×3 and the two additional-seed stability runs are effectiveness-only controls: we omit AST TED and choose the earliest candidate when failed candidates tie in pass rate, which cannot change PR, RR, or IR.

5.6 Metrics and Statistical Analysis

For M instances, let c_m be the current program and $\hat{c}_m$ the selected result. $\text{Pass}(c) \in [0, 1]$ is the test pass fraction. Pass Rate (PR) averages $\text{Pass}(\hat{c}_m)$; Repair Rate (RR) averages $\mathbf{1}[\text{Pass}(\hat{c}_m) = 1]$; Improvement Rate (IR) averages $\mathbf{1}[\text{Pass}(\hat{c}_m) > \text{Pass}(c_m)]$.

For successfully repaired, parseable programs, $\text{TED}_{B\to F}$ measures AST distance from current to fixed code and $\text{TED}_{F\to O}$ from fixed code to the recorded accepted oracle. RQ2 uses $\text{TED}_{F\to T}$, where T is the identical adapter-specific recorded target shared by Full and Current configurations.

We report Wilson 95% intervals for RR and IR. Paired RR comparisons use two-sided exact McNemar tests, with Holm correction within each planned comparison family. Because trajectories from one problem are dependent, our primary robustness analysis resamples problems as clusters 10,000 times with seed 2027 and reports intervals for paired RR, PR, and IR differences. We additionally report a problem-balanced estimand that gives every problem equal weight, with a problem bootstrap interval. Paired TED differences use 10,000 instance bootstrap resamples on jointly repaired inputs. Unless stated otherwise, percentages are absolute percentage points.

6 Results

6.1 RQ1: Repair Effectiveness

Table 4 summarizes RQ1 and RQ5. On Seen problems, ZPDPatch repairs 593 of 997 programs (59.5%), compared with 306 (30.7%) for Zero-shot. Their Wilson RR intervals are [56.4, 62.5] and [27.9, 33.6], respectively. ZPDPatch alone repairs 320 instances, whereas Zero-shot alone repairs 33 (exact McNemar $p < 10^{-50}$). More importantly for dependent trajectories, the 28.8-point paired RR gain remains under problem-cluster bootstrap [24.4, 33.1]; the equal-problem-weight gain is 24.1 points [19.4, 28.7]. PR increases by 10.23 points [8.38, 12.15] and IR by 23.3 points [18.9, 27.5] under the same clustered analysis.

LSGen achieves the highest Seen execution correctness: 88.0% PR, 80.2% RR, and 82.4% IR. It alone repairs 280 instances, while ZPDPatch alone repairs 73 (exact McNemar $p < 10^{-28}$); the clustered RR difference is −20.8 points [−25.0,−16.5]. The tradeoff is patch size. ZPDPatch's mean $\text{TED}_{B\to F}$ is 21.10, versus 27.25 for Zero-shot and 88.27 for LSGen. Among jointly repaired parseable instances, ZPDPatch reduces TED by 8.95 relative to Zero-shot (95% CI [5.41, 13.05]) and by 42.68 relative to LSGen (95% CI [37.77, 47.79]).

Table 5 shows that the gains are not marginal exchanges: against Seen Zero-shot, ZPDPatch has 320 unique repairs versus 33; against LSGen, it has 73 versus 280. LSGen has higher coverage, but neither success set contains the other.

PR and IR confirm that the RR gain is accompanied by partial behavioral improvement. Patch distance separates repair from replacement: mean current-to-fix TED is 22.6% below Zero-shot and 76.1% below LSGen on each method's successful set, while the jointly repaired paired analyses above establish the same direction without success-set confounding.

Answer to RQ1. ZPDPatch substantially outperforms a trajectory-prompted zero-shot model and produces smaller successful patches. Same-problem retrieval remains much stronger in repair

Table 4. Main repair results. PR, RR, and IR are percentages; TED is computed on successfully repaired, parseable programs.

Split	Method	PR	RR	IR	$\text{TED}_{B\to F}$	$\text{TED}_{F\to O}$
Seen	Zero-shot	76.3	30.7	43.7	27.25	27.67
	LSGen	**88.0**	**80.2**	**82.4**	88.27	83.23
	ZPDPatch	86.5	59.5	67.0	**21.10**	**21.45**
Unseen	Zero-shot	75.3	62.0	68.8	17.46	15.80
	ZPDPatch	**84.5**	**73.2**	**79.2**	**11.48**	**12.54**

Table 5. Paired complete-repair outcomes. "Only ZPDPatch" and "only comparator" are the discordant pairs used by McNemar tests. Counts are derived from the same instances as Table 4.

Split	Comparator	Both	Only ZPDPatch	Only comparator	Neither
Seen	Zero-shot	273	320	33	371
Seen	LSGen	520	73	280	124
Unseen	Zero-shot	139	44	16	51

coverage, so ZPDPatch is complementary to, rather than a replacement for, a correct-solution database.

6.2 RQ2: Trajectory Conditioning

Table 6 shows no consistent advantage for Full Trajectory. On Seen Strict examples, Full raises PR by 0.7 points but lowers RR and IR by 0.7 and 1.3. On Seen Progress, it lowers PR, RR, and IR by 1.0, 0.4, and 0.6. On Unseen data, Full is worse for Strict but better for Progress. Target TED is also mixed: Full is closer to the recorded target for Seen Progress and Unseen Strict, but farther in the other conditions. None of the four paired RR differences is significant after Holm correction (minimum adjusted $p = 0.313$).

Each Full–Current pair holds training examples, targets, architecture, hyperparameters, and evaluation inputs fixed; only earlier submissions are removed and position leakage reset. Direction changes across conditions, clustered RR intervals include zero (Unseen Strict reaches zero at its upper endpoint), and target TED is mixed. RQ1 can therefore establish complete-system utility while RQ2 rejects history serialization as a stable causal explanation.

Answer to RQ2. Providing full submission history does not consistently improve repair over current code alone. RQ1 therefore cannot be interpreted as evidence for a causal history-conditioning effect; it measures the complete learned and sequential system.

6.3 RQ3: Adapter Specialization

Table 7 exposes a coverage-size tradeoff. Answer has the highest PR, RR, and IR, but also the largest mean patch at TED 20.22. Progress has lower coverage but the smallest patch at TED 15.85. Strict and Progress have statistically indistinguishable RR (adjusted $p = 0.857$). Answer repairs 5.2 points more than Progress and 5.5 points more than Strict; problem-cluster intervals are [2.5, 7.9] and [2.9, 8.0], respectively. The ordered increase in patch size is consistent with broader targets, but Strict does not form a clearly distinct coverage regime.

Coverage specialization is weak between Progress and Strict, but patch-scope specialization is ordered: their mean current-to-fix TEDs are 15.85, 17.45, and 20.22 through Answer, while Answer

Table 6. RQ2 trajectory-conditioning results. PR, RR, and IR are percentages. Current and Full use identical examples and targets.

Split	Adapter	Input	N	PR	RR	IR	$\text{TED}_{F\to T}$
Seen	Strict	Current	2,151	56.8	31.2	54.6	40.23
	Strict	Full	2,151	57.5	30.5	53.3	41.05
	Progress	Current	2,674	58.4	26.3	49.7	36.23
	Progress	Full	2,674	57.4	25.9	49.1	33.15
Unseen	Strict	Current	322	66.7	56.2	71.4	20.20
	Strict	Full	322	65.4	53.4	70.8	19.27
	Progress	Current	378	65.7	52.1	66.4	17.63
	Progress	Full	378	66.5	53.7	68.3	18.16

Table 7. RQ3 individual-adapter results on 997 Seen instances.

Adapter	PR	RR	IR	$\text{TED}_{B\to F}$	$\text{TED}_{F\to O}$
Progress	73.4	44.8	52.0	**15.85**	**15.89**
Strict	74.8	44.5	51.2	17.45	18.38
Answer	**77.5**	**50.1**	**57.5**	20.22	20.61

repairs 5.2–5.5 points more. Oracle distance follows the same pattern. These are distributional effects rather than decoding constraints; hard non-regression enters only through selection.

Answer to RQ3. The supervision rules induce different repair-size behavior: direct answer supervision produces the broadest and most successful individual adapter, while progress supervision produces the smallest successful patches. Verdict-only Strict is not clearly separated from Progress in repair coverage.

6.4 RQ4: Sequential Escalation

Independent-policy escalation reaches 59.5% RR, 9.4 points above the strongest individual adapter, Answer; its problem-cluster interval [7.6, 11.3] excludes zero. Gains over Progress and Strict are 14.6 [12.3, 17.0] and 14.9 [12.8, 17.1] points, respectively. The increase appears in PR and IR as well, so the portfolio is not trading complete repairs for weaker partial outputs.

Table 8 expands control flow. Progress accepts 447 inputs, Strict accepts 60 more, and 490 reach Answer. The resulting 997 + 550 + 490 = 2,037 generations average 2.04 per input, avoiding 954 (31.9%) of an always-three budget.

Returned-source counts exceed early stops because final selection can recover an improving failed candidate. The unchanged program is returned 329 times, making certified abstention a substantial behavior rather than a corner case.

Answer to RQ4. Execution-guided escalation improves both complete repair and partial improvement over every single adapter. Repeating Answer three times reaches 55.2% RR, so additional calls to the strongest individual policy recover 5.0 points over Answer once but remain 4.31 points below the heterogeneous independent portfolio. That diversity gain survives problem-cluster resampling [2.16, 6.47] and paired testing (91 portfolio-only versus 48 repeated-Answer-only repairs; McNemar ($p=0.000333$)); PR (+2.45 [1.38, 3.56]) and IR (+4.61 [2.49, 6.75]) agree. Keeping policies conditioned on authentic observations additionally gains 2.91 RR points over generated feedback [0.81, 5.01],

Table 8. RQ4 stage flow and final selector provenance on 997 Seen inputs. “Reached” counts generated candidates; “accepted here” counts early stops. Selector provenance includes early stops and final selection.

Stage / source	Reached	Accepted here	Returned by system
Progress	997	447	485
Strict	550	60	79
Answer	490	86	104
Unchanged fallback	–	–	329

Table 9. RQ4 sequential escalation on 997 Seen instances. “Candidates” is the mean number generated.

Configuration	PR	RR	IR	Candidates
Progress	73.4	44.8	52.0	1.00
Strict	74.8	44.5	51.2	1.00
Answer	77.5	50.1	57.5	1.00
Answer×3	84.1	55.2	62.4	1.98
Generated Feedback	84.9	56.6	64.6	2.05
Independent Policies	**86.5**	**59.5**	**67.0**	2.04

with PR (+1.65 [0.62, 2.71]) and IR (+2.41 [0.11, 4.71]) improvements. Thus neither extra calls to one policy nor synthetic prompt-state updates explain the independent portfolio’s gain. Early stopping avoids a fixed three-generation cost, although the sequence still averages roughly twice the generation work of an individual adapter.

6.5 RQ5: Problem Generalization

On 250 Unseen instances, ZPDPatch obtains 84.5% PR, 73.2% RR, and 79.2% IR, versus 75.3%, 62.0%, and 68.8% for Zero-shot. ZPDPatch alone repairs 44 cases and Zero-shot alone repairs 16 (McNemar $p = 0.000394$). Problem-cluster intervals exclude zero for RR (+11.2 points [5.15, 17.26]), PR (+9.18 [5.27, 13.34]), and IR (+10.4 [4.94, 15.92]); the equal-problem-weight RR gain is 10.47 points [3.98, 16.78]. ZPDPatch also reduces mean successful-patch TED from 17.46 to 11.48. Among 119 jointly repaired parseable instances, its patch is 6.14 TED units smaller on average (95% CI [3.13, 9.56]).

Heterogeneous policies also outperform repeated use of Answer on Unseen data: 73.2% versus 68.4% RR, with 15 portfolio-only and three Answer×3-only repairs (McNemar $p = 0.00754$) and a +4.8-point problem-cluster interval [1.09, 9.21]. PR improves by 3.34 points [1.12, 5.98]. Against generated feedback, the independent system gains 1.2 RR points [−0.80, 3.31] and 1.54 PR points [0.14, 3.34]; the RR interval includes zero, while the PR interval does not. Thus policy diversity generalizes across problem holdout, whereas the smaller benefit of preserving authentic prompt state is supported more strongly by Seen RR and Unseen PR than by Unseen RR alone.

The 44 favorable versus 16 unfavorable discordant pairs against Zero-shot show complementary successes rather than dominance. Absolute Unseen RR is higher for both methods, but the split was defined by trajectory availability rather than randomized difficulty. The valid estimand is the paired within-Unseen difference, which remains positive without same-problem training.

Answer to RQ5. ZPDPatch’s advantage over Zero-shot persists on problems completely excluded from adapter training. Because the Unseen group deliberately contains lower-resource problems, its higher absolute RR must not be interpreted as evidence that unseen problems are easier or that familiarity harms repair.

6.6 RQ6: Guarantees and Robustness

We executed an independent certificate checker over every canonical training example. Answer's 40,454 examples all contain one failed source and an accepted terminal target. The checker evaluated 19,981 retained Strict transitions and 29,920 retained Progress transitions; every transition satisfied its defining relation. All 16,973 unique Strict target events occur among the 21,416 Progress target events, with zero missing events. These exhaustive counts connect Propositions 1–3 to the materialized datasets rather than only to pseudocode. Eight executable property tests additionally exercise rank descent, terminal closure, inclusion, assistance-horizon nesting, selection, effectiveness-only tie handling, and all policy permutations over randomized finite trajectories.

The selection certificate likewise reports zero violations on all 997 Seen and 250 Unseen inputs. In every case, selected pass rate is at least current pass rate; the minimum difference is exactly zero because abstention is allowed. Thus, Proposition 4 is both a general property of Algorithm 2 and an audited property of the reported outputs. The problem-cluster and problem-balanced intervals reported in RQ1–RQ5 further show that the main positive conclusions do not depend on treating trajectories from one assignment as independent.

Complete retraining with seeds 2027/2028/2029 yields Seen RR 59.5/57.6/59.0% (mean 58.7, SD 1.0 points) and Unseen RR 73.2/74.0/70.4% (mean 72.5, SD 1.9). Mean PR/IR are 86.2/66.2% on Seen and 83.5/78.4% on Unseen. Every Seen run exceeds the earlier Generated Feedback configuration's 56.6% RR. This cross-run comparison bounds QLoRA variation but does not replace the paired seed-2027 comparison, which alone holds trained adapters fixed while changing candidate dependence.

Finally, we freeze the three independently generated candidate sets and replay all six policy orders (Table 10). Because candidate membership is fixed, coverage is identical at 59.5%; the replay isolates which accepted candidate is encountered first and how many candidates are inspected. Progress–Strict–Answer has the lowest mean policy breadth (1.391 on a 1–3 scale) and the smallest mean successful-patch TED (21.10). Answer-first orders reduce generation calls to 1.92–1.93 but select substantially broader policies and patches. Progress–Answer–Strict is a nearby alternative with fewer calls but higher breadth and TED. The deployed order therefore occupies the conservative endpoint of a measured cost–intervention frontier, rather than being chosen because Progress has the highest standalone RR.

The 59.5% replay is the final independent-policy system result: every adapter candidate is conditioned on the same authentic trajectory, as required by Algorithm 2. The six-order replay changes only early-stop precedence, so it is also an exhaustive empirical certificate of Proposition 5. Separately generated-feedback and repeated-Answer controls test whether synthetic prompt-state updates or repeated use of the strongest single policy explain the portfolio gain.

Answer to RQ6. The execution-evidence and non-regression propositions hold without violations in the complete artifacts. Problem-clustered inference preserves the principal effects, three full training runs bound seed variation, and exhaustive order replay identifies the selected order as the least-broad, smallest-patch endpoint among fixed-candidate portfolios.

6.7 Cross-RQ Synthesis

Together, the RQs locate the gain in two components: authentic trajectories supply ordered offline supervision, and execution controls online escalation and abstention. Target-matched retraining shows that these relations can be distilled without requiring a long history at deployment; Progress and Strict differ more in patch scope than coverage; problem holdout, cluster resampling, and independent retraining preserve the system advantage; and certificates connect observed outputs to the formal relations. This decomposition identifies a stronger mechanism than prompt-level "trajectory conditioning": trajectories define learnable reachability structure.

Table 10. Static replay of all policy orders on identical Seen candidates. P, S, and A denote Progress, Strict, and Answer. Breadth assigns policy levels 1, 2, and 3; TED is over parseable repairs.

Order	RR	Generations	Breadth	TED
P–S–A	59.5	2.043	**1.391**	**21.10**
P–A–S	59.5	1.976	1.460	21.16
S–P–A	59.5	2.046	2.039	22.16
S–A–P	59.5	1.989	2.153	22.45
A–P–S	59.5	**1.924**	2.715	22.30
A–S–P	59.5	1.934	2.793	22.63

7 Discussion

7.1 What Produces the Observed Improvement?

Adapter training matters because ZPDPatch beats a zero-shot model that already receives trajectory context; sequential execution matters because composition beats every member; and target-matched current-code adapters retain the relational benefit. The practical consequence is favorable: expensive history serialization is not required for every deployment even though authentic histories remain essential for constructing the ordered targets. Counterfactual histories paired with identical current code would further test when path information changes the appropriate repair rather than only its supervision source.

7.2 Coverage Versus Patch Size

LSGen demonstrates the value of same-problem correct programs, reaching higher coverage with mean TED over four times ZPDPatch's. Smaller edits can preserve structure but do not prove pedagogical superiority. The measured tradeoff is software-level: retrieval maximizes coverage in repository-mature settings, whereas adapters make more conservative changes and remain applicable to held-out problems.

7.3 Implications for Educational Use

ZPDPatch produces programs, not hints; deployment should expose diffs or derive a hint ladder rather than overwrite work. Its fallback prevents presenting observed-test regressions as help. A learner study measuring transfer, time, and help seeking is still required to establish educational benefit; the present contribution is a repair and feedback-authoring substrate.

8 Threats to Validity

Construct validity. Passing the available tests is a proxy for semantic correctness and may admit overfitting. We reduce this risk by requiring the recorded oracle to pass the same suite, but hidden tests could change absolute rates. AST TED measures structural change, not readability, conceptual difficulty, or educational value. "Reachable" means observed later in a trajectory; it does not imply that the generated transition lies within an individual learner's ZPD. We therefore avoid claims about learning outcomes.

Internal validity. Dataset construction, verdict ordering, normalization, runner limits, and candidate selection can affect results. We use one runner and shared outcome cache across methods, preserve full trajectories after filtering, and compare paired outputs on identical instances. LSGen required infrastructure adaptation; although its retrieval and generation logic is retained, our integration may differ from its original environment. Greedy decoding removes sampling variance, and three full QLoRA runs quantify training-seed variation, but other hyperparameters and stochastic decoding remain untested.

Data leakage and dependence. Unseen problems are completely excluded from training and retrieval. Seen splits share problem identities by design, and users may occur across splits; the benchmark is not user-held-out. We address assignment-level dependence with problem-cluster bootstrap and an equal-problem-weight estimand. Residual dependence through users who solve multiple problems remains possible; a complementary user-held-out study would test personalization transfer rather than problem transfer.

External validity. The study covers Python solutions from Project CodeNet, one 7B code model, one GPU, and programming-contest-style tasks. Results may not transfer to multi-file projects, other languages, industrial defects, private autograders, or novice courses with different submission behavior. The Unseen set consists of lower-volume problems by construction and should not be treated as a random sample of all new assignments.

Conclusion validity. We define explicit metric families, correct planned RR comparisons, and repeat primary effect estimates with problem clusters. TED intervals remain conditional on joint successful repair and should not be generalized to failures. The current model family and testcase suite still delimit the measured effects; additional architectures and hidden tests would test whether the ordering transfers.

Ethical considerations. The study analyzes a publicly released code dataset and performs no intervention with learners. We do not attempt to re-identify users, and derived evaluation reports need only pseudonymous identifiers. Nevertheless, source-code histories can carry authorship signals; a public artifact should minimize identifiers and follow Project CodeNet's distribution terms.

9 Conclusion

We presented ZPDPatch, an execution-guided APR framework trained from authentic student submission trajectories. Nested execution-evidence orders expose monotonic testcase progress and verdict improvement, while terminal closure supplies accepted targets. Their finite-chain, inclusion, selection-safety, and minimum-escalation properties are proved and mechanically verified. Independent-policy composition raises Seen repair rate from 50.1% for the strongest individual adapter to 59.5%, while early stopping limits generation to 2.04 candidates on average. The complete system outperforms a trajectory-prompted zero-shot model on Seen and problem-held-out Unseen data under problem-clustered inference and produces smaller successful patches, although same-problem retrieval achieves higher coverage. Three full training runs retain this behavior with Seen and Unseen RR standard deviations of 1.0 and 1.9 points.

Target-matched current-code adapters retain the benefit without serializing prior submissions. Trajectories therefore act as a source of executable reachability relations that can be distilled into model parameters, while the certified portfolio converts complementary relations into minimum-breadth repairs.

10 Data Availability

An anonymized package will provide code, prompts, scripts, split/configuration records, outputs, and a manifest binding 143 records (2.29 GB) to revision `aa908288a397` with SHA-256 digests and row counts. Raw Project CodeNet data will not be redistributed; acquisition instructions, deterministic regeneration commands, and checksums will be provided.

References

Vincent Aleven and Kenneth R Koedinger. 2000. Limitations of student control: Do students know when they need help?. In *International conference on intelligent tutoring systems*. Springer, 292–303.

Dongwook Choi, Jinseok Heo, and Eunseok Lee. 2021. Automated Feedback Generation for Multiple Function Programs. In *2021 28th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 582–583.

Dongwook Choi and Eunseok Lee. 2025. Automated Feedback Generation for Programming Assignments Through Diversification. In *2025 IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 230–241.

Irene-Angelica Chounta, Patricia Albacete, Pamela Jordan, Sandra Katz, and Bruce M McLaren. 2017. The "Grey Area": A computational approach to model the Zone of Proximal Development. In *European conference on technology enhanced learning*. Springer, 3–16.

Michael Cole, Vera John-Steiner, Sylvia Scribner, and Ellen Souberman. 1978. Mind in society. *Mind in society the development of higher psychological processes. Cambridge, MA: Harvard University Press* (1978).

Albert T Corbett and John R Anderson. 1994. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User modeling and user-adapted interaction* 4, 4 (1994), 253–278.

Zhenlong Dai, Zhuoluo Zhao, Hengning Wang, Xiu Tang, Sai Wu, Chang Yao, Zhipeng Gao, and Jingyuan Chen. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. *arXiv preprint arXiv:2601.08545* (2026).

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems* 36 (2023), 10088–10115.

Aritra Ghosh, Neil Heffernan, and Andrew S Lan. 2020. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2330–2339.

Sumit Gulwani, Ivan Radiček, and Florian Zuleger. 2018. Automated clustering and program repair for introductory programming assignments. *ACM SIGPLAN Notices* 53, 4 (2018), 465–480.

Hasnain Heickal and Andrew Lan. 2024. Generating feedback-ladders for logical errors in programming using large language models. *arXiv preprint arXiv:2405.00302* (2024).

Jinseok Heo, Hohyeon Jeong, Dongwook Choi, and Eunseok Lee. 2023. Referent: Transformer-based feedback generation using assignment information for programming course. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. IEEE, 101–106.

Yang Hu, Umair Z Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2020. Re-factoring based program repair applied to programming assignments. In *Proceedings-2019 34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE, 388–398.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).

Harshit Joshi, José Cambronero Sanchez, Sumit Gulwani, Vu Le, Gust Verbruggen, and Ivan Radiček. 2023. Repair is nearly generation: Multilingual program repair with llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 5131–5140.

Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2018. A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education (TOCE)* 19, 1 (2018), 1–43.

Fang Liu, Zhenwei Liu, Qianhui Zhao, Jing Jiang, Li Zhang, Zian Sun, Ge Li, Zhongqi Li, and Yuchi Ma. 2024. Fastfixer: An efficient and effective approach for repairing programming assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 669–680.

Tom Murray and Ivon Arroyo. 2002. Toward measuring and maintaining the zone of proximal development in adaptive instructional systems. In *International conference on intelligent tutoring systems*. Springer, 749–758.

Susanne Narciss. 2008. Feedback strategies for interactive learning tasks. In *Handbook of research on educational communications and technology*. Routledge, 125–143.

Pedro Orvalho, Mikoláš Janota, and Vasco M Manquinho. 2025. Counterexample guided program repair using zero-shot learning and maxsat-based fault localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 649–657.

Tung Phung, José Cambronero, Sumit Gulwani, Tobias Kohn, Rupak Majumdar, Adish Singla, and Gustavo Soares. 2023. Generating high-precision feedback for programming syntax errors using large language models. *arXiv preprint arXiv:2302.04662* (2023).

Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. 2015. Deep knowledge tracing. *Advances in neural information processing systems* 28 (2015).

Thomas W Price, Yihuan Dong, and Dragan Lipovac. 2017. iSnap: towards intelligent tutoring in novice programming environments. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on computer science education*. 483–488.

Ruchir Puri, David S Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladmir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, et al. 2021. Project codenet: A large-scale ai for code dataset for learning a diversity of coding tasks. *arXiv preprint arXiv:2105.12655* 1035 (2021).

Lianne Roest, Hieke Keuning, and Johan Jeuring. 2024. Next-step hint generation for introductory programming using large language models. In *Proceedings of the 26th Australasian Computing Education Conference*. 144–153.

Ke Wang, Rishabh Singh, and Zhendong Su. 2018. Search, align, and repair: data-driven feedback generation for introductory programming exercises. In *Proceedings of the 39th ACM SIGPLAN conference on programming language design and implementation*. 481–495.

Ruiwei Xiao, Xinying Hou, and John Stamper. 2024. Exploring how multiple levels of GPT-generated programming hints support or disappoint novices. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–10.

Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F Bissyandé, and Shunfu Jin. 2024. Cref: An llm-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 882–894.

He Ye, Matias Martinez, Xiapu Luo, Tao Zhang, and Martin Monperrus. 2022. Selfapr: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

Jooyong Yi, Umair Z Ahmed, Amey Karkare, Shin Hwei Tan, and Abhik Roychoudhury. 2017. A feasibility study of using automated program repair for introductory programming assignments. In *Proceedings of the 2017 11th joint meeting on foundations of software engineering*. 740–751.

Jialu Zhang, José Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2022. Repairing bugs in python assignments using large language models. *arXiv preprint arXiv:2209.14876* (2022).

Jialu Zhang, José Pablo Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2024. Pydex: Repairing bugs in introductory python assignments using llms. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 1100–1124.

Qianhui Zhao, Fang Liu, Li Zhang, Yang Liu, Zhen Yan, Zhenghao Chen, Yufei Zhou, Jing Jiang, and Ge Li. 2024. Peer-aided repairer: Empowering large language models to repair advanced student assignments. *arXiv preprint arXiv:2404.01754* (2024).